

Lec 09 Practical Exercise - 2

Lec 09 Practical Exercise - 2

Lecture	09 / 63 + intro
Duration	29:49
Captions	English (auto-generated)
Words	4,682
Video	1zLfwJgBnOo
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello all, welcome to the hands-on series on fundamentals of generative AI and large language models theory and practice. In today's session, we'll be implementing the ensemble deep learning models. Ensembling in machine learning or deep learning has its own advantages. In case of ensembling, what we do is we take the predictions of different models, make the models run parallelly, take their prediction and either perform averaging of those predictions or then stack those one over the other and get the final prediction. In today's implementation, we are taking three different CNN models that is convolutional neural networks having its own uh customized way of uh predictions and then do we do the averaging of the

[00:52] predictions. So let us go through the implementation as the usual pro procedure we first import the necessary libraries. So importing TensorFlow as TF. TensorFlow is a deep learning framework which is required for building the neural networks. From tensorflow keras we import the data sets layers as well as models. So importing of data sets is necessary uh because we are dealing with some of the data sets which are readily available either in UCI repository or Kaggle. So we are taking one such uh data set which is the SIART 10 data set which is Canadian Institute for Advanced Research data set having 10 different classes of data comprising of vehicle data, animal

data and the bird data which has 10 different classes. So we

[01:40] are importing one of uh such a data set then importing the different layers. So in order to build a convolutional neural network we need a set of input layer, different hidden layers and the output layer. So all these could be built with the help of a function which is layers and in order to construct or build or train the models we need a function called as models. So for visualization purpose we make use of the matplotlib lib.pipplot which is in short uh called as plt. importing the numerical Python to deal with the arrays or numbers we uh import it as NP and from scikitlearn classification matrix we import the confusion matrix as well as confusion matrix display. So this kind of matrix or a matrix display is required in order

[02:29] to show the performance of the model uh wherein we try to compare the actual uh output with the predicted output of the model. So this is all about importing the necessary libraries. Next coming to loading the data set. So uh we are uh making use of the SIP 10 data set. We load that data set into the training and the test set. So we have some portion of the training set and uh some portion of the test set. So we define two variables in each of this training as well as test. So X train Y train which comprises of the input training and output training set. uh y x test and y test which comprises of the input test set and the output test set. So since we are dealing with images and images has two

[03:17] to 56 different intensity values which can vary from 0 to 255. What what happens to the model is it has to learn different values maybe a smaller value which is 0 1 2 3 and when we take on the higher side on the brighter side of the image it may have to learn up to 255 uh values. So in order to making the learning easier for the model what we try to do is normalize the value. So normalize in the range maybe minus1 to 1 or 0 to 1. So in order to do that we take this kind of uh normalization wherein we normalize the pixel values in the range 0 to 1. So we take all the pixel values uh minimum pixel value will be 0. So 0 by 255 the minimum value is 0. Once again normalized minimum value the highest pixel value is 255. When we

[04:10] make uh 255 divided by 255 we get 1. So the intermediate values between 0 to 255 will anyways lie in the range 0 to 1. So we have seen this example in the previous session wherein we try to build a CNN model and we have seen with the help of example how we try to normalize the different pixel values. Now coming to the 10 different classes present in the SIP data set. The 10 different classes are aeroplane, automoto, bird, cat, deer, dog, frog, horse, ship and the truck. So it comprises of various vehicle names, animals as well as birds. Now let us print the training and the test set. So X train shape and X uh X test shape. So shape is a function which will help in getting the length of the training as well as the test set. So

[05:03] in the output we can see that uh training data comprises of 50,000 training samples or the uh image samples which are of the size 32 + 32 + 3. So the first 32

refers to the width of the image. Next 32 refers to the height of the image and three refers to the number of channels. So since it is a RGB image, there are three channels which refers to red, green and blue. Coming to the test data, there are 10,000 test samples. Once again the size is $32 + 32 + 3$ referring to the width, the height and the number of channels.

[05:42] Now coming to defining the different models. An ensemble model is a combination of n models that is n could be two or more. So let us see with an example what is ensemble. So ensemble can happen serially or parallelly. In our implementation we are going for the parallel method. So we define one model we call it as model one. We define another model we call it as model two. And the third model which is called as model three.

[06:18] We have training data, test data both given to model 1, two as well as three. So training data plus the test data. This combination is given to model one as well as two and the third model. So we have prediction one which is obtained for the first model. Prediction two which is obtained for the second model. Similarly prediction three obtained for the third model. Say we take index zero. Say for example we want the prediction for index zero. That is we have x_{test} and we have y_{test} . We want this for the zeroth data.

[07:10] Now there is some prediction made by model one. Say for example, model one predicts the zeroth test data as truck. Model 2 predicts it as same automobile class and model 3 predicts it as truck. Then what happens is we have to take the majority of the classes. So two different models have taken the prediction has made the prediction as truck. So the majority prediction is truck. So the final prediction will be truck.

[07:47] Final prediction will be truck. So what is the benefit of having three different models is we are making sure through every prediction that there is no major misclassification. But if you take model one alone, model two alone or model 3 alone, the number of misclassifications may be more. So what we are trying to do is we are trying to improve the performance by doing this kind of an ensemble. So in case of model one we have used a customized CNN with some input layers, the convolution layers, max pooling layers and the dense layer. In case of model 2, we have once again custom, we have used the customized CNN but we have done this batch normalization. So batch normalization is a regularization

[08:40] technique which makes the model more generalized and tries to avoid the overfitting condition. And in model 3 once again it is a customized CNN itself but we have used a CNN along with the dropout concept. So dropout is one more way of regularizing the model. That is we are trying to reduce the number of neurons or effect or impact of certain neurons so that they will not memorize one neuron will not memorize the contents or the images or the patterns of the other neurons. So

with the help of dropout we are reducing the training we are reducing the impact of certain neurons. So this is model 1 2 and three.

[09:22] So let us build these three models and let us see what are the predictions done by these three models and how exactly ensembling takes place here. So coming to defining the model one we are making use of the uh definition here uh define or create the CNN one. So the it is a sequential model because the layers are present sequentially one after the other. So we have the first convolution layers with the 32 uh different neurons or 32 different uh neurons here 3 +3 filter activation function is relu shape as mentioned earlier 32 + 32 + 3. So 32 width 32 height and three channels soon after which we have the max pooling layer of the uh filter size 2 + 2. So in the previous uh hands-on session we have

[10:15] already seen the importance of this convolution as max pooling but to summarize here convolution helps in feature extraction. So as we go from the first layer to the second layer the number of filter size has increased. So basic uh filtering happens with the help of 32 different filters. A basic filtering happens here. low-level filtering and deep extraction of features or high level extraction of features happens with 64 different filters. So this is where the model will be able to do the feature extraction and do some prediction for us. So this is about having two different layers of convolution. Now coming to the max pooling layer whatever feature map we obtain from the convolution layer here

[11:00] or the convolution layer in the second case that from that feature map we are trying to get the maximum values. So maximum values is nothing but the maximum pixel intensity that we are going to derive. So that refers to the uh that that refers to the feature which is very much important or the pixel which is highly contributing to the prediction or the classification. So that's why we are using the max polling and then uh we are having this uh image size image size is 32 + 32 + 3 we have three different channels here. So and as well as it is a matrix of 32 width and 32 height. Now this 32 + 32 will be converted into 1D array with the help of this flatten function. Why this is required is whatever feature map that we

[11:48] get after max pooling that has to be convert that that is considered as features feature x1 x2 x3 so on up to the number of uh uh the number of features that we get from the previous layer. So those features have to be multiplied by the weights and then added to the bias in order to get some raw score. From the raw score we have to obtain the probability and that probab probability will uh attribute or will uh will correspond to the particular class and that happens to the class label or the output label for the given input image. That is why we need to convert this 2D array or 3D array into a flattened layer which is a 1D array. And at the output layer we have 128 neurons as uh which comprises the dense layer

[12:38] with the activation function relu. And finally one more output layer since we

have 10 classes in our uh uh input data. This corresponds to 10 different neurons. So there should be 10 neurons because there are 10 classes in the given data. And final activation function is the softmax activation function because we have more than uh two classes here. That's why we try to use softmax in order to get the probability as well. So this is the first CNN model that we have built. Now coming to the second CNN model again a sequential model with the layers convolution layer batch normalization activation function used as ReLU and max pooling. Similarly the same combination repeated once again so that there is

[13:25] highle filtering. So this is first convolution layer. This is the second convolution layer followed by all the functions which have taken place in the first layer also batch normalization activation and max pooling followed by flattened layer which is used to convert 3D array into uh or 2D array into 1D array and finally the output layer consisting of 128 neurons and finally the 10 neurons because 10 neurons correspond to 10 different classes. The now the only difference between this convolution uh neural network when compared to model one is the use of this batch normalization. So what exactly is done in this batch normalization is see we have the 2 +2 pulled output or the output from the rail activation

[14:24] function. See if we have this and this may be two this may be 10 this is three and this is four. So for every value that is present or input value or the intensity value that is present we find the normalized value. So X norm will be equal to the actual input minus the mean divided by the standard deviation. So if I have to find the value normalized value for this two then this will be 2 minus the average of these four values that is $2 + 10 + 3 + 4$ divided by the four values and then the standard deviation. So this will help me in getting the normalized value. What is the benefit of normalized value is it will always lie in a particular range and uh whenever we want a gshian kind of distribute uh distributed data or a

[15:15] normally distributed data uh which which follows the shape of a bell curve most of the data lying uh within a certain standard deviation and very few outliers present this kind of normalization is very much important. So when we have a normally distributed data the outliers will be very uh very much less and because the outliers are less the less number of mclassifications take place. So that is a benefit of having the batch normalization so that we have a data in a proper range or a defined range.

[15:50] So this is the second model that we have defined. Now coming to the third model once again sequential uh layer because we have the convolution followed by max pooling once again convolution followed by max pooling layer and then the flattened layer with the dense layer. Now at the output side what we try to do is uh this 128 neurons what we have are now deactivated. All the neurons will will not be used. uh what we are trying to do is we are cutting short 50% of the neurons. See we have hidden layers.

[16:28] Assume we have this kind of hidden layers. Every neuron can be connected to every other neuron in this way. So when it is connected to every other neuron, it is called as a fully connected network. So what happens is if you consider this as hidden layer 1, this is hidden layer two and these neurons as A B C D E F always we know that we are going to get some data uh to D from the neuron A. Similarly E will get from neuron A, F will get from neuron A. So when this kind of learning happens all the neurons will try to memorize the data. So memorizing the data is nothing but memorizing the patterns. So when it tries to memorize the patterns only of the training data in case of the unseen data or a validated data it is not able

[17:21] to it will not be able to understand the patterns. So this is where the majority of the misclassifications take place. So what we can do is the same neurons if I take A B C D E F A we will try to give a connection to D and A from A we will try to give a connection to F. From B let us give a connection to E and B to D. Maybe C to E and C to A. So with this we are trying to reduce the connections or remove the connection from one neuron to another neuron so that they don't memorize the data. So this condition is called as a dropout condition. Now out of 10 connections if we remove exactly five connection it is 50% of the connections which has been removed. So that is called as 50% dropping out of the

[18:13] connections. So what we are doing is we are trying to cut off the 50% of the connection so that we get a more generalized uh model. So this is all about the third CNN model wherein we are trying to give importance to the dropouts. So we have defined three different uh CNN models. In the first one it was just a a customized CNN a same CNN model which we had used in the previous uh uh previous hands-on. the second model CNN with batch normalization and the third uh model with CNN and dropout. So when we have these three models let us uh build one N symbol model. So model one we have created CNN. In model two we have the second CNN. Model 3 we have the third CNN. So we have let us put together all

[19:03] three models in this way and compile that in a variable called as model. So we will try to use a optimizer called as Adam loss uh as sparse categorical cross entropy. Why sparse? Because the 10 different classes are given 10 different unique integers. Maybe automobile called as uh may be called as a zeroth index. Truck may be called as first index. Uh the car may be called as third index and so on. So we have 10 different index varying from 0 to 9. Since they are integers, we are we are supposed to use the sparse categorical cross entropy. In case they are not uh integers and they have more than two classes, we can just use the categorical cross entropy. And if we have only two classes, we are uh

[19:50] we can be just adjusting it with the binary cross entropy. And now coming to the metrics, we can either measure accuracy or loss. So in this implementation, let us use the accuracy matrix. Now coming to training of the model. So we have three different models. So three different models have to be trained. Model one, model two

and model 3. So training happens on the training data which comprises of X train as well as Y train. Let the number of epochs be five.

[20:20] We can also uh we can always have more number of epochs. It is again based on user to use or how many epochs you want to use, how much time you can invest on uh learning as well as execution. So for simplicity we have used only five epochs and let the batch size be equal to 64. So we know that there are 50,000 training samples. Those training samples need not be used in a single stretch. Instead what we can do is updation of the weights instead of happening at the end of 50,000 samples we can divide the training samples into different batches.

[20:56] So first 64 samples we will take we will do the prediction. we will find the loss uh during the back propagation we can find the weights after updating that weights that new weight is used for u for the next 64 samples again do some kind of prediction uh do the uh find the loss and updation of the weights so that the second weight is uh pass it on to the third one so likewise we have 64 different uh bats we have the bat size as 64 and now let Let us divide this uh training data into validation as well.

[21:34] So let the 20% of the training data go into validation. That's why we have validation split as point2. So the same thing is used for model 2 as well as model 3 because we have to train three different models. So now when we see the summary of this training we have epoch 1 to 5 run for the first model. Again we have epoch 1 to 5 which is run for the second model. Similarly 1 to 5 run for the third model because we have defined three different models. So at the end of fifth epoch let us see what is the validation accuracy.

[22:10] So 67% is a validation accuracy for model one. Then we see here 68% is a model uh validation accuracy for model 2 and for model 3 we have 61%. So batch normalization has really uh worked or it has it it is very effective compared to the other two models. So now coming to predictions. So averaging the predictions. So model one is used to predict on X test. Model two used to predict the X test. Similarly model 3 prediction done on the X test.

[22:50] So these predictions are saved in prediction 1, two as well as three. Now that prediction we are considering here taking the sum of it and then averaging nothing but majority voting. So whichever uh votes are the highest that particular class will be considered for the final prediction. So this is the nse symbol prediction here. So this is saved the ny symbol prediction is saved in a variable called as n symbol spread.

[23:20] Now n symbol spread like we will have 10 different probabilities because we have 10 different class. So whichever probability is the highest we want the index of that particular probability that index will correspond to the particular class maybe automoto truck car and so on. That's why we always mention argument maximum. So we want the that index for which the probability is high that has to be reflected. So that will be stored now in a new variable called as y prediction. So this is y prediction

which which is got by ensemble of prediction.

[24:00] Now let us display the nse symbol accuracy. So uh we define this uh particular function from matrix. So from scikit learn matrix let us import the accuracy score. So this accuracy score is got by obtaining the uh is is got by comparing y test as well as y prediction. So let us save in a variable called as accuracy. When we try to print that accuracy uh with the print statement and symbol accuracy we see that the accuracy is 71% 7114.

[24:36] So which is higher when compared to the individual accuracy. We have seen that individual accuracy corresponded to 67% 68 as well as 61%. So when we have done that for uh many number of epochs and for all the test samples the ensemble accuracy is slightly higher which is 71. Now coming to the last part that is the confusion matrix. So in order to have a confusion matrix we need to import the confusion matrix as well as confusion matrix display. So confusion matrix is a matrix wherein we try to compare the uh true label with the predicted label. So we want the y test which comprises of the actual label and the y bread which comprises of the n symbol prediction prediction done by three different

[25:21] models. So let us display the confusion matrix based on uh what is defined in this variable that is cm. Let the color map be blue and let us show this confusion matrix. So what is happening in the matrix is on the x-axis or on the rows we have the true label and on the x is equal to 1 we have the predicted label that is on the columns we have the predicted label. So we have zeroth class, first 2, 3rd, fourth, likewise up to ninth one. The index varying from 0 to 9 which uh constitutes the 10 different classes.

[26:01] Similarly 0 to 9 which comprises of 10 different classes. Unlike our previous hands-on, we have not shown the different uh classes here the naming or the labels given here just to make sure that it works even this way even uh it can work with the help of integers and that is the reason we have used the sparse categorical cross entropy because um a model or the computer will not understand the naming or the label given instead they will understand only the numbers or the inteious associated with it. So now coming to the matrix here whatever we see on the diagonal are the true classifications that is when zero class is predicted as zeroth class itself then that kind of prediction that kind of classification is shown here. So

[26:50] we have 8.5 such classifications when uh class number one is being predicted as class number one itself. Likewise we have 839 such instances. Similarly, class 2 being predicted as class 2 itself 547 such predictions. So diagonal elements are the correct classifications. Now coming to the non-diagonal elements. If I take this example of 19 class 0 being predicted as class one. So likewise we have 19 such examples. If I take example of 73 here.

[27:26] So class number two being predicted as class number five. So we have 73

such examples. So whatever nondagonal elements we have when we sum it up we get the uh number of mclassifications. So number of mclassification will tend to the loss loss that is given by the model and the hues of blue uh are given here. Lesser the number the lighter shade of blue and the larger the number we can see the darker blue here. So with the help of confusion matrix it is very much evident that the number of correct classifications are higher than the number of mclassification and that is why we have the uh prediction accuracy as 71% which is more than 50% of the data.

[28:16] So in this uh implementation we have seen the ensembling deep learning models. We have taken three different models that is a customized convolutional neural network. The second model was customized CNN with the batch normalization technique where wherein batch normalization tries to have a normally distributed data which works on uh zero mean and uh the standard deviation of one that is trying to give us the regularized model or a more generalized model. And the third model we used a customized CNN with certain dropouts. So in this example we have used 50% of the dropouts that is 50% of the neurons were deactivated during training. So when these three models are made to run uh parallelly and only their

[29:04] predictions are been compared and average voting is been done we have got these predicted output. So these predicted output or the accuracy was 71% which is slightly uh higher when compared to the individual predictions. So we have seen the individual predictions as 61% 67 or 68 which is less when compared to the ensembling model. So ensembling model will try to give the lesser number of mclassification because we are trying to consider uh two or more models for uh prediction. So this is where the performance of each of the models will increase or performance of the ensemble model will increase. Thank you.